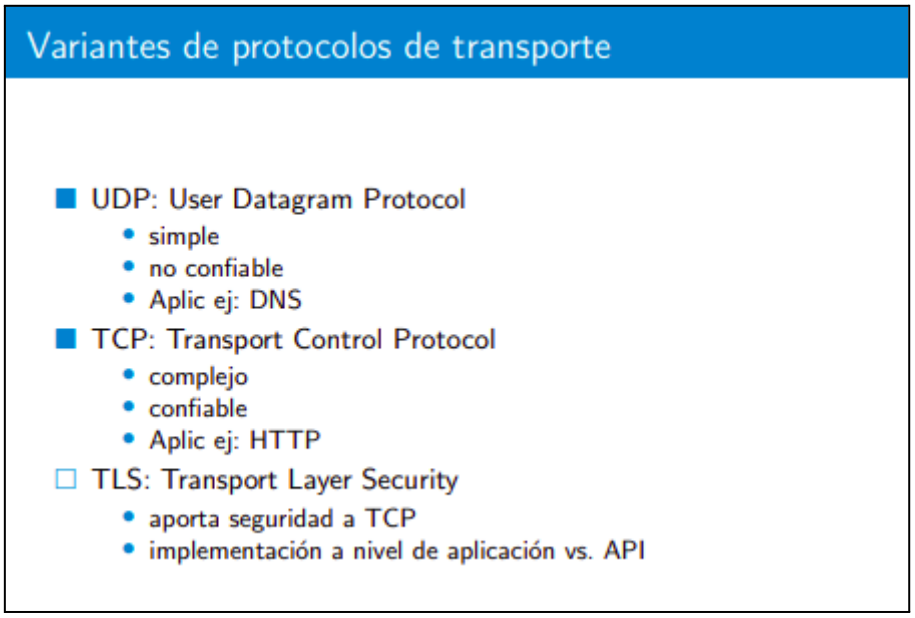
Ejercicios que desarrollamos en la clase del jueves 20 de Marzo

5. ¿Cuáles son algunas diferencias entre TCP y UDP?



to do a little more than nothing! At the very least, the transport layer has to provide a multiplexing/demultiplexing service in order to pass data between the network layer and the correct application-level process.

UDP, defined in [RFC 768], does just about as little as a transport protocol can do. Aside from the multiplexing/demultiplexing function and some light error checking, it adds nothing to IP. In fact, if the application developer chooses UDP instead of TCP, then the application is almost directly talking with IP. UDP takes messages from the application process, attaches source and destination port number fields for the multiplexing/demultiplexing service, adds two other small fields, and passes the resulting segment to the network layer. The network layer encapsulates the transport-layer segment into an IP datagram and then makes a best-effort attempt to deliver the segment to the receiving host. If the segment arrives at the receiving host, UDP uses the destination port number to deliver the segment's data to the correct application process. Note that with UDP there is no handshaking between sending and receiving transport-layer entities before sending a segment. For this reason, UDP is said to be *connectionless*.

DNS is an example of an application-layer protocol that typically uses UDP.

Veamos qué más dice Kurose sobre UDP:

Now you might be wondering why an application developer would ever choose to build an application over UDP rather than over TCP. Isn't TCP always preferable, since TCP provides a reliable data transfer service, while UDP does not? The answer

is no, as some applications are better suited for UDP for the following reasons:

- *Finer application-level control over what data is sent, and when [...]*
- *No connection establishment [...] For this reason, a server devoted to a particular application can typically support many more active clients when the application runs over UDP rather than TCP.*
- *No connection state [...]*
- *Small packet header overhead. The TCP segment has 20 bytes of header overhead in every segment, whereas UDP has only 8 bytes of overhead.*

Veamos también qué relaciones existen entre TCP/UDP y algunos protocolos de la capa de aplicación:

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Secure remote terminal access	SSH	TCP
Web	HTTP, HTTP/3	TCP (for HTTP), UDP (for HTTP/3)
File transfer	FTP	TCP
Remote file server	NFS	Typically UDP
Streaming multimedia	DASH	TCP
Internet telephony	typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP
Name translation	DNS	Typically UDP

Figure 3.6 ♦ Popular Internet applications and their underlying transport protocols

6. ¿Por qué TCP y UDP no tienen mecanismos de cifrado?

Por diseño: ninguno de los dos especifica mecanismos de cifrado propios en los protocolos, y delegan (implícitamente) estas funciones a la capa de aplicación.

FOCUS ON SECURITY

SECURING TCP

Neither TCP nor UDP provides any encryption—the data that the sending process passes into its socket is the same data that travels over the network to the destination process. So, for example, if the sending process sends a password in cleartext (i.e., unencrypted) into its socket, the cleartext password will travel over all the links between sender and receiver, potentially getting sniffed and discovered at any of the intervening links. Because privacy and other security issues have become critical for many applications, the Internet community has developed an enhancement for TCP, called **Transport Layer Security** (TLS) [RFC 5246]. TCP-enhanced-with-TLS not only does everything that traditional TCP does but also provides critical process-to-process security services, including encryption, data integrity, and end-point authentication. We emphasize that TLS is not a third Internet transport protocol, on the same level as TCP and UDP, but instead is an enhancement of TCP, with the enhancements being implemented in the application layer. In particular, if an application wants to use the services of TLS, it needs to include TLS code (existing, highly optimized libraries and classes) in both the client and server sides of the application. TLS has its own socket API that is similar to the traditional TCP socket API. When an application uses TLS, the sending process passes cleartext data to the TLS socket; TLS in the sending host then encrypts the data and passes the encrypted data to the TCP socket. The encrypted data travels over the Internet to the TCP socket in the receiving process. The receiving socket passes the encrypted data to TLS, which decrypts the data. Finally, TLS passes the cleartext data through its TLS socket to the receiving process. We'll cover TLS in some detail in Chapter 8.

Entre otras razones de diseño, podemos pensar por ejemplo en el costo computacional que hubiese implicado para un procesador de 1974 (año de creación de TCP) hacer el “setup” de la sesión TLS/SSL.

Teniendo en cuenta que un core virtualizado de intel puede típicamente hacer unas 350 operaciones de handshake del protocolo SSL con llaves de 2048 bits, el costo para un procesador de los 70s hubiese sido prohibitivo!

Veamos un extracto de la RFC 5246, que define el protocolo TLS (Transport Layer Security) como un protocolo de la capa de aplicación (por encima de protocolos de transporte como por ejemplo TCP):

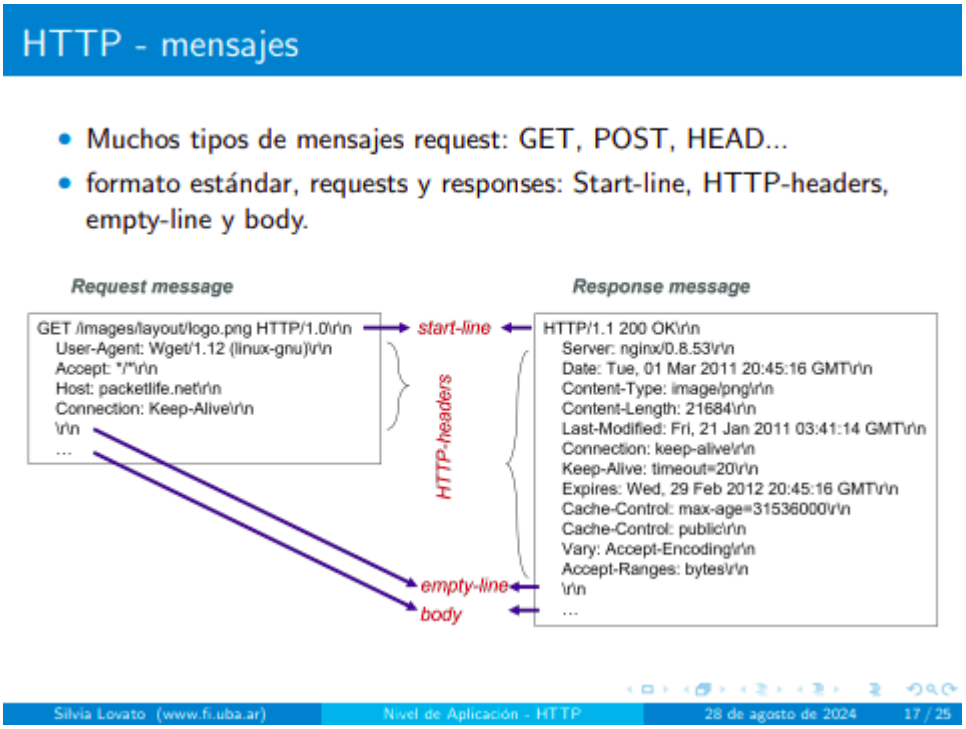
1. Introduction

The primary goal of the TLS protocol is to provide privacy and data integrity between two communicating applications. The protocol is composed of two layers: the TLS Record Protocol and the TLS Handshake Protocol. At the lowest level, layered on top of some reliable transport protocol (e.g., TCP [TCP]), is the TLS Record Protocol. The TLS Record Protocol provides connection security that has two basic properties:

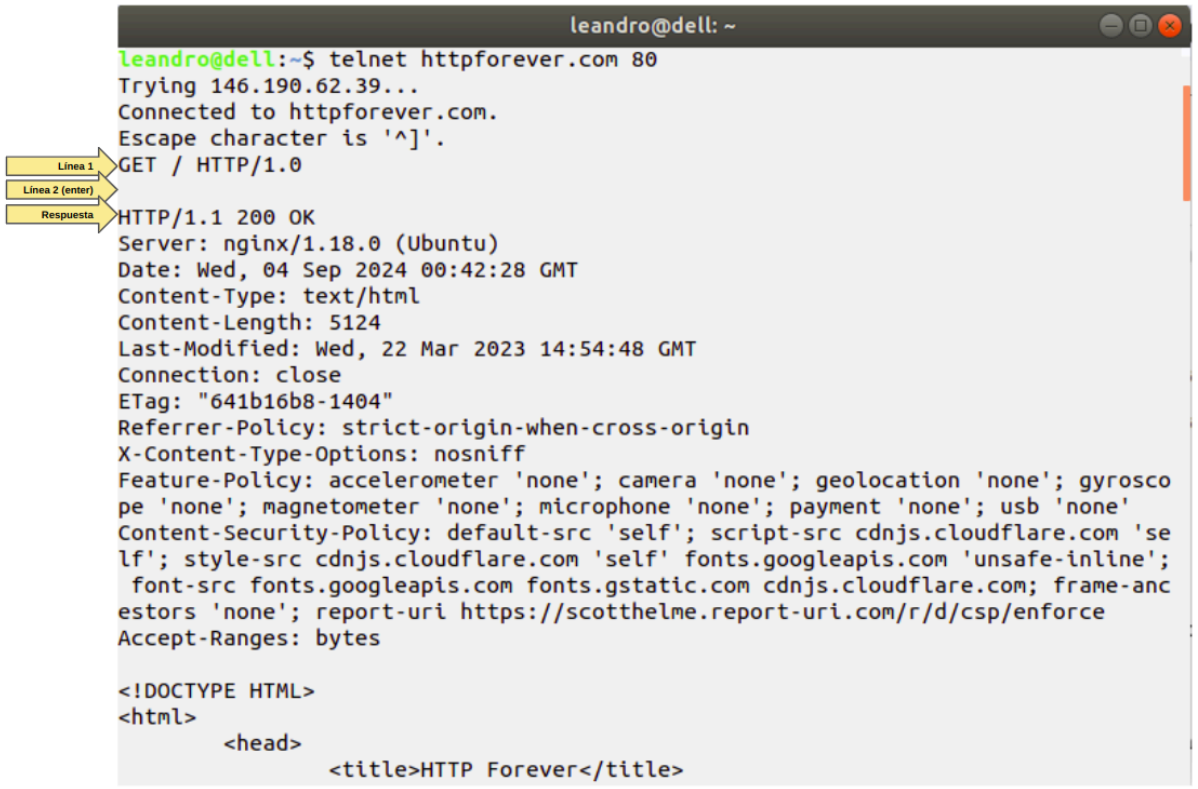
- The connection is private. Symmetric cryptography is used for

8. ¿Cuál es la diferencia entre una conexión HTTP persistente y una conexión no persistente?

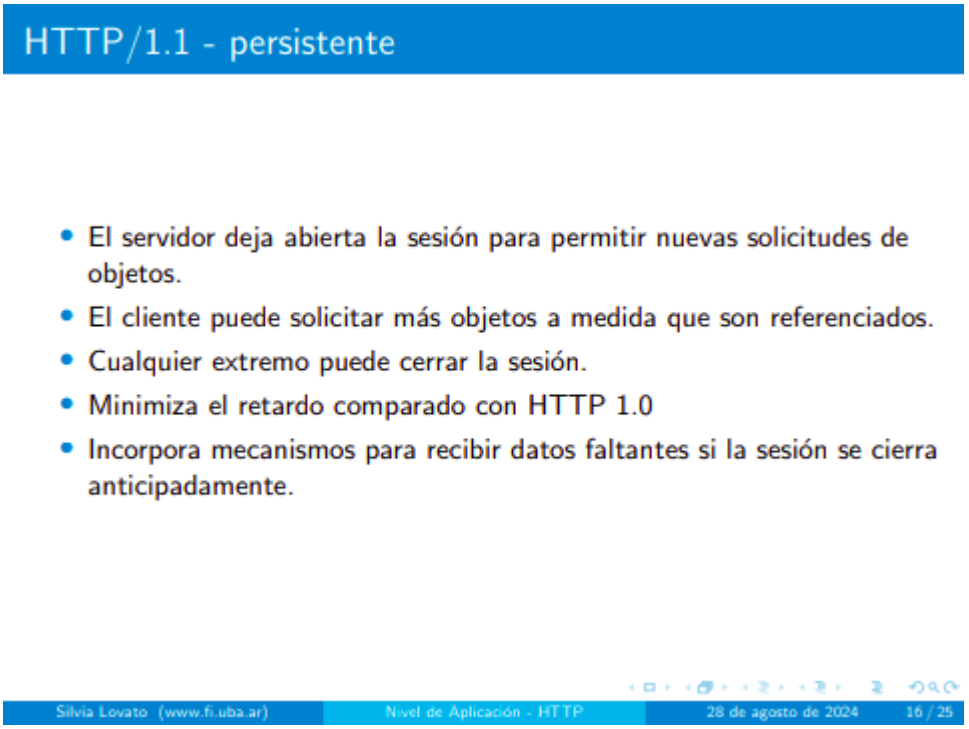
Recordemos primero lo visto en la teórica:

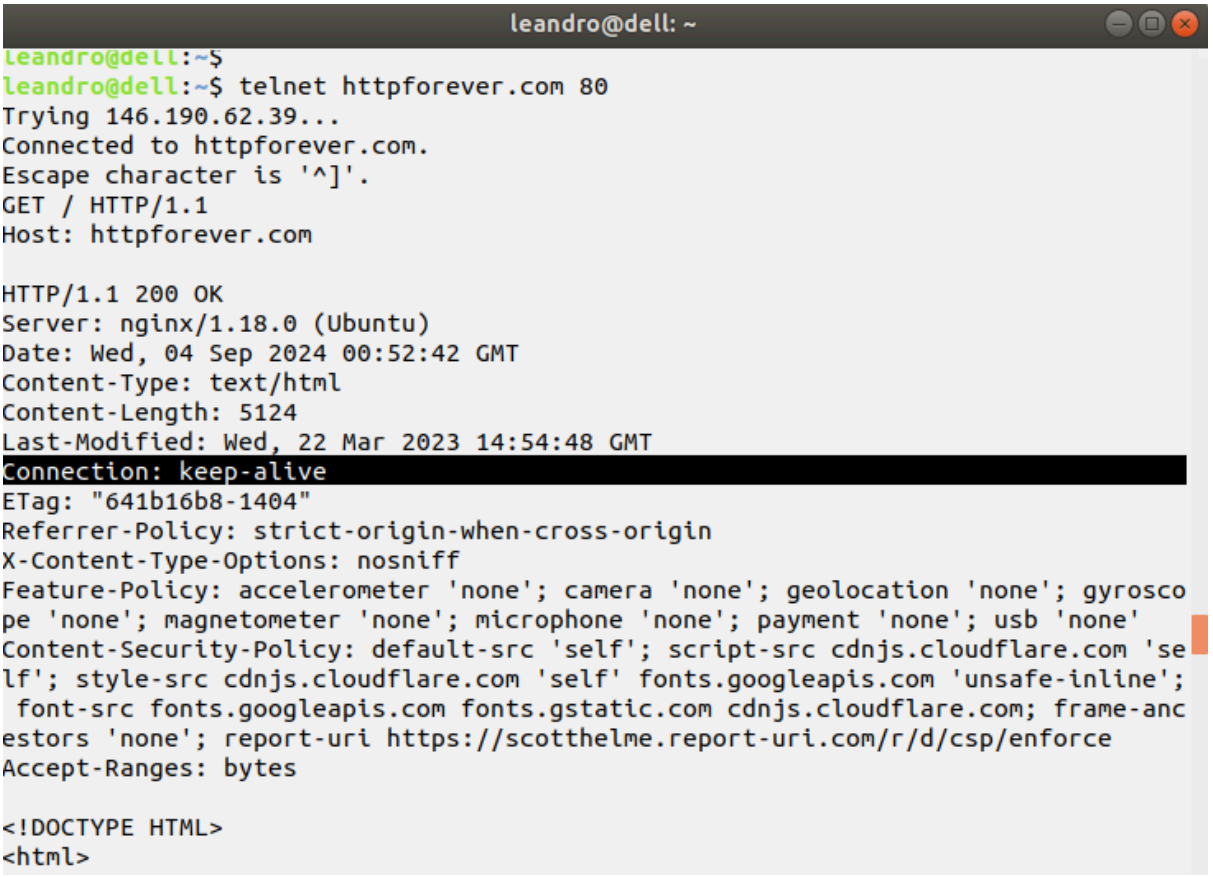


Vamos a conectarnos port HTTP a http://httpforever.com/ y enviar una solicitud HTTP 1.0 (no persistente).



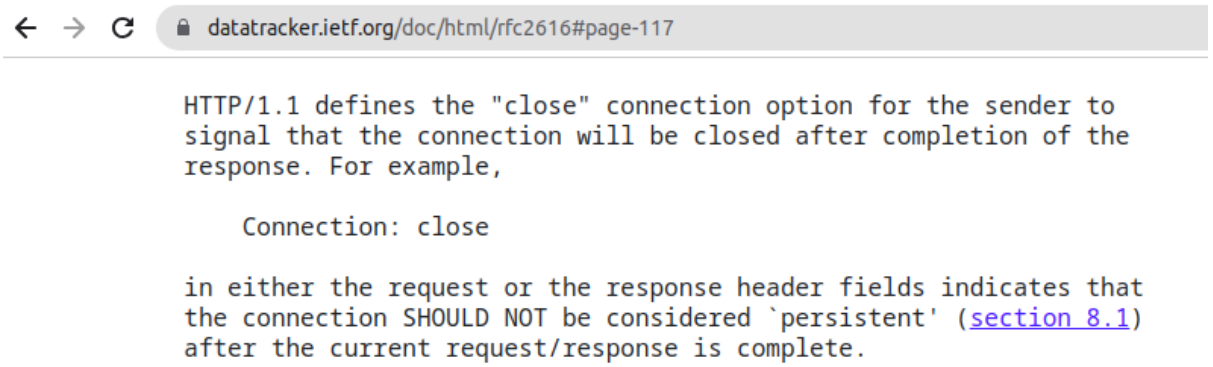
Ahora vamos con una solicitud HTTP/1.1 y conexión persistente. Primero, revisemos nuevamente lo visto en la teórica:





Conexión no persistente en HTTP/1.1

Recordemos qué dice la RFC 2616 al respecto:



Veamos qué ocurre al incluir el header `Connection: close` en la solicitud HTTP/1.1:



…



👆 Aquí vemos que, al incluir el encabezado **`Connection: close`** en el header de la solicitud HTTP/1.1, le estamos indicando al servidor web que no debe considerar la sesión HTTP como persistente: efectivamente, la respuesta que nos envió el servidor web refleja esto ya que vemos que el Apache:

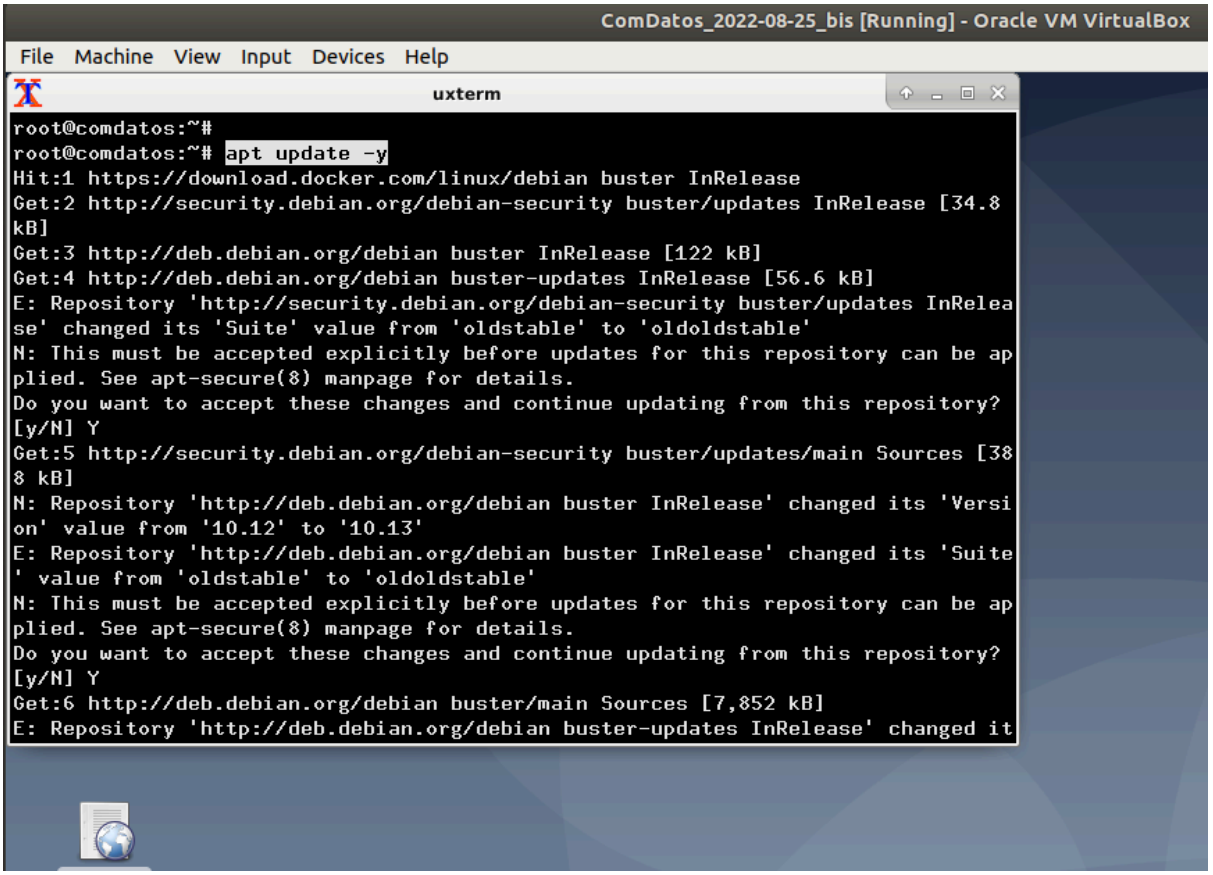
1. Incluyó el **`Connection: close`** en su respuesta para indicarnos que la conexión no debe interpretarse como persistente.
2. Cerró la conexión de la capa de transporte al terminar de enviarnos la respuesta a nuestra solicitud. Esto lo podemos ver en la parte de la salida del comando telnet, que imprime "**Connection closed by foreign host**" en cuanto detecta el cierre de la misma.

12. Haga telnet a un servidor web y envíe un mensaje de solicitud multilínea. Incluya en el mensaje de solicitud la línea de encabezado If-modified-since: para forzar un mensaje de respuesta con el código de estado 304 No modificado.

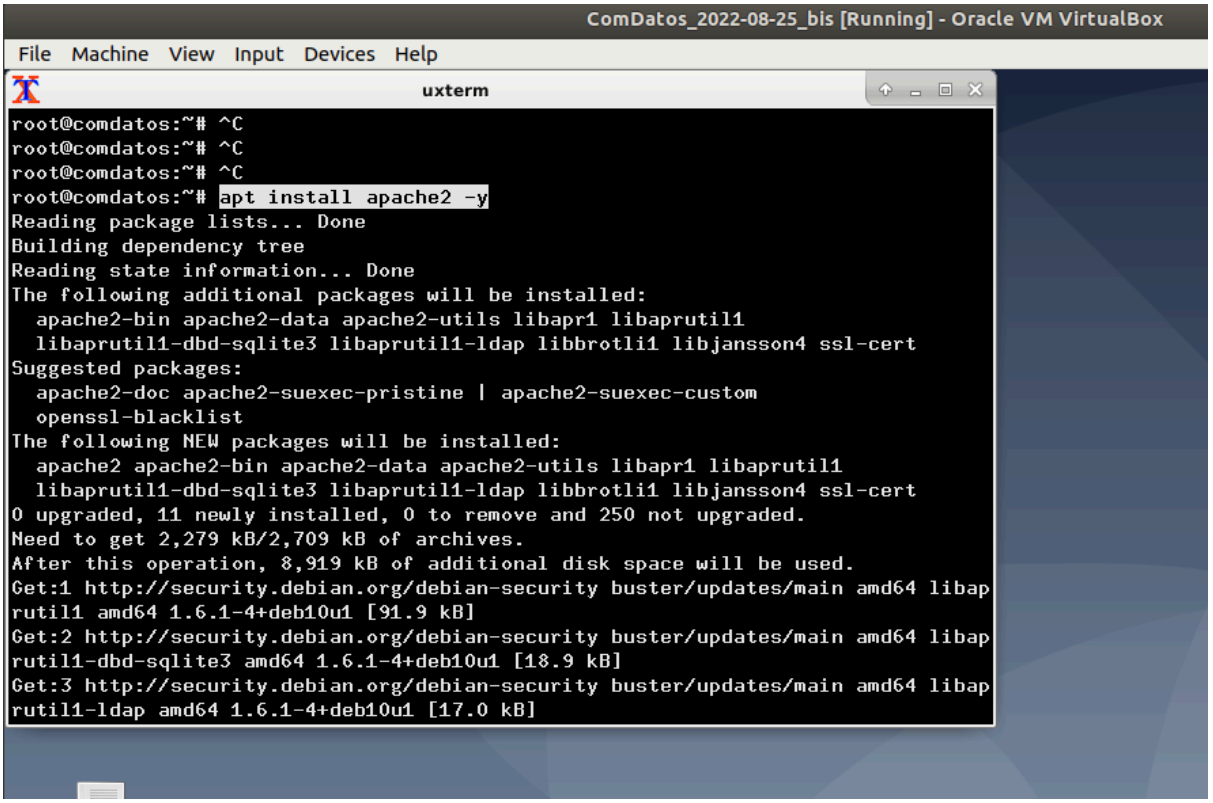
Para resolver este ejercicio vamos a usar la VM de VirtualBox. Lo primero que vamos a tener que hacer es instalar el servidor web apache en la propia VM, ya que no viene instalado por defecto.

Para esto, debemos ejecutar los siguientes comandos:

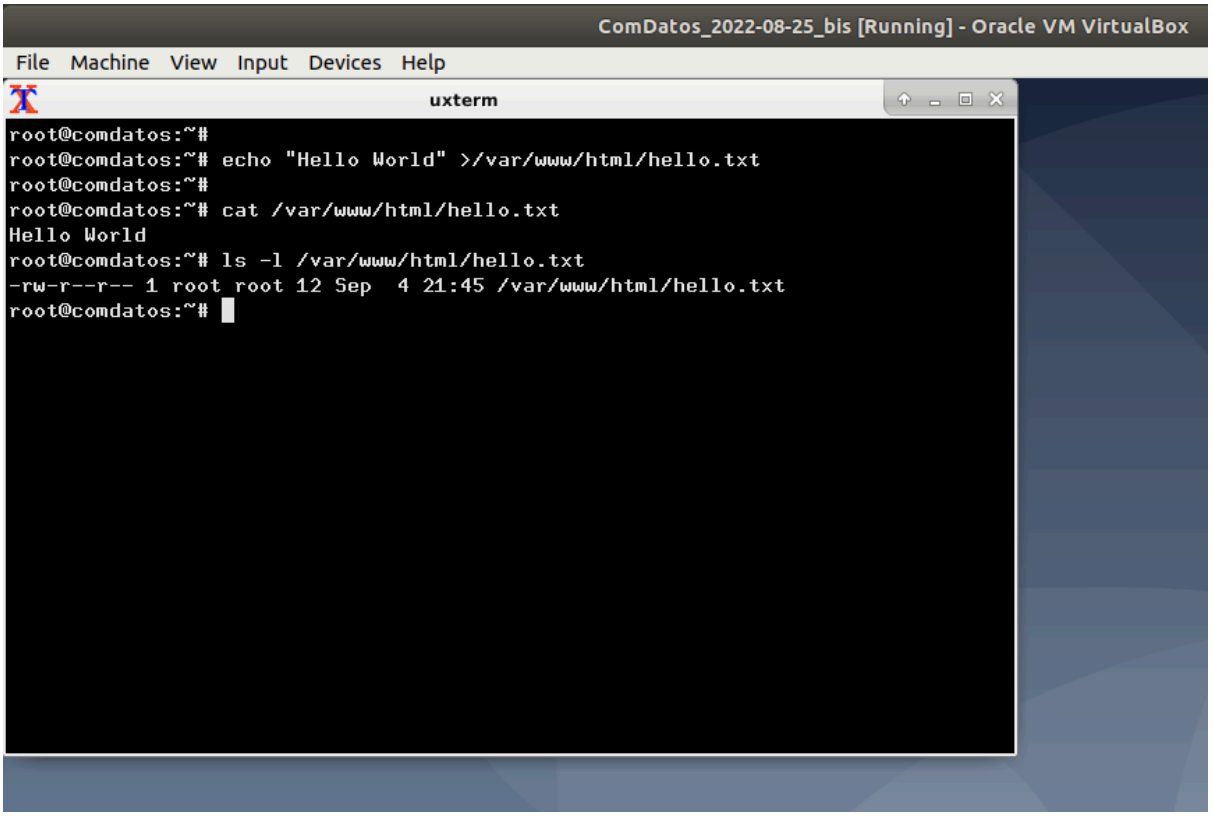
apt update -y



apt install apache2 -y

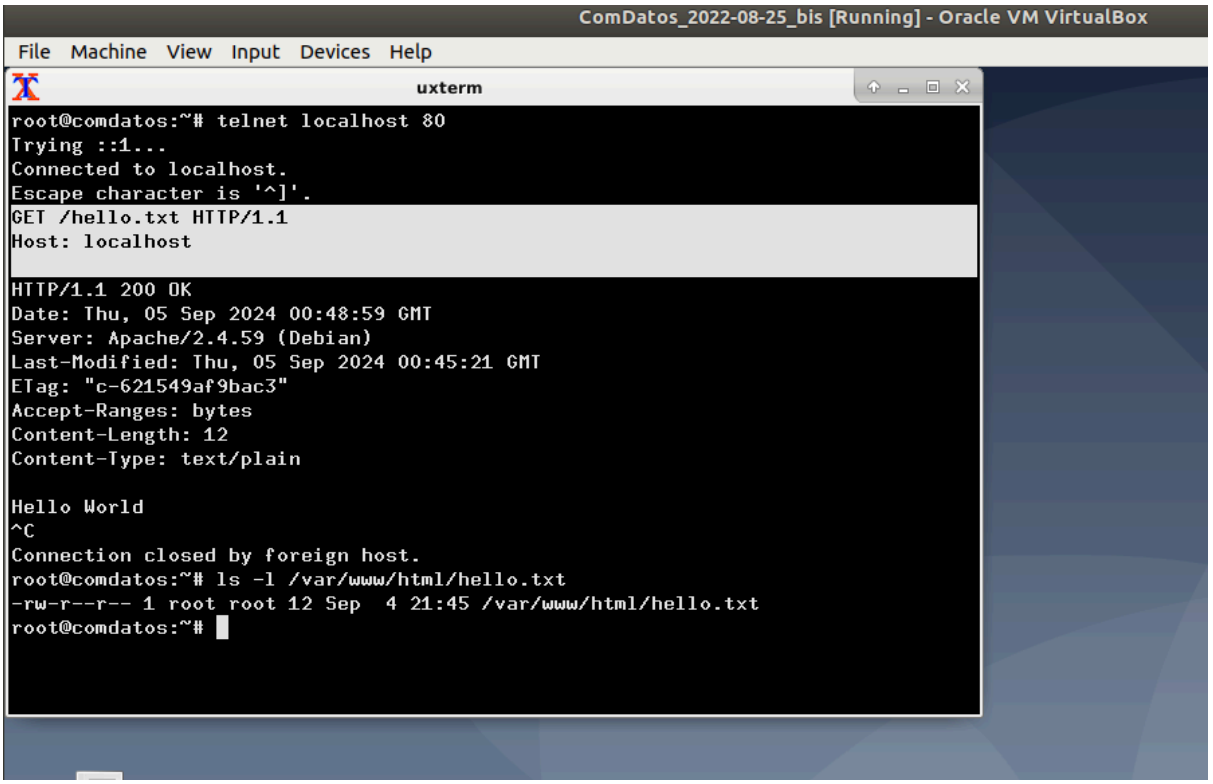


Luego, vamos a crear un archivo de prueba en /var/www/html/, que es el directorio que el Apache tiene configurado como DocumentRoot (*)



(*) DocumentRoot es el directorio de base al cual el Apache va a referir todas las solicitudes. Es decir, más abajo, cuando solicitemos “/hello.txt”, el Apache va a mapear esta solicitud al sistema de archivos en la ubicación “/var/www/html/hello.txt”.

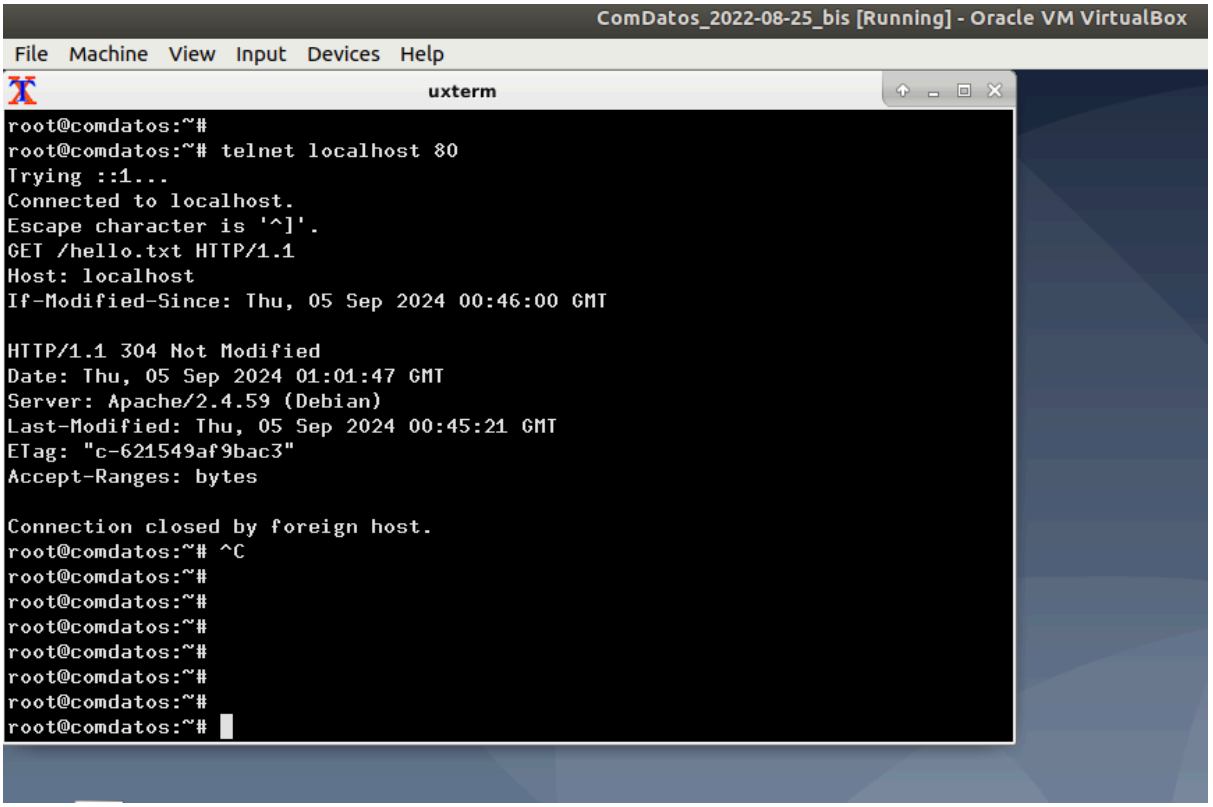
Luego, lo que vamos a hacer es conectarnos al servidor web (Apache) que está corriendo en la propia VM y hacer un GET para que nos transfiera este archivo:



Aquí podemos ver varias cosas:

- El header Last-Modified generado por el servidor web que nos indica la fecha de modificación del archivo que pedimos (y que como podemos ver ejecutando el “ls -l “ coincide con la fecha de modificación en el sistema de archivos)
- Content-Length y Content-Type para describir el tamaño en bytes y tipo MIME del contenido (inferido por el servidor web)

Ahora vamos a ejecutar una nueva solicitud, pero enviando el header If-Modified-Since con un valor posterior a la fecha de modificación del archivo en el sistema de archivos (en mi caso, voy a incrementar en un minuto esa fecha, es decir, usaré 21:46 como valor para If-Modified-Since)



Observar el código HTTP usado en la respuesta (304 en este caso)

Links mencionados durante la clase del 5 de Septiembre

(sólo para contexto de los que estuvieron presencialmente ese día)

FSDB(8) FreeBSD System Manager's Manual
https://man.freebsd.org/cgi/man.cgi?query=fsdb